

Technik Programista

Testowanie i dokumentowanie aplikacji

Markdown

Format dokumentacji programisty

Nagłówki, listy, kod, tabele, linki – i dlaczego README.md, a nie README.docx

Klasa 4 | rok szkolny 2026/2027 | System: Windows

Spis treści

Spis treści

Spis treści.....	2
1. Co to jest Markdown.....	3
1.1 Gdzie go użyjesz	3
2. Składnia – to, co naprawdę potrzebne	4
2.1 Nagłówki	4
2.2 Akapity i podział linii.....	4
2.3 Wyróżnienia	4
2.4 Listy	4
2.5 Kod	5
2.6 Linki i obrazy.....	5
2.7 Tabele.....	5
2.8 Cytat i linia pozioma.....	5
3. README – szablon na projekt.....	6
4. Zasady	7
5. Zadania	8
Zadanie 1: README od nowa	8
Zadanie 2: Notatki z materiału 08	8
Zadanie 3: Zrzut ekranu.....	8
Zadanie 4: Issue	8
Zadanie 5: Plik dla agenta	8

1. Co to jest Markdown

Markdown (pliki `.md`) to sposób zapisu formatowanego tekstu **zwykłymi znakami** w pliku tekstowym. Zamiast klikać „pogrubienie” w Wordzie, piszesz **`**tekst**`**. Zamiast wstawiać nagłówek – `# Tytuł`. Plik da się czytać bez żadnego programu, a GitHub, VS Code, PyCharm, Discord, Notion i większość narzędzi programistycznych renderują go do ładnego widoku.

Dlaczego programiści piszą dokumentację w Markdown, a nie w Wordzie:

- **tekst = Git.** Plik `.md` ma historię zmian, diff, pull requesty – jak kod. Docx to binarny blob; Git widzi tylko „zmienił się”.
- **leży obok kodu.** `README.md` w repo jest pierwszą rzeczą, którą GitHub pokazuje. Nikt nie szuka „dokumentacja_final_v3.docx” na dysku.
- **bez formatowania do popsucia.** Nie ma stylów, marginesów, czcionek. Jest treść.
- **wszędzie to samo.** Ten sam plik wygląda tak samo na GitHubie, w edytorze, w wiki, w generatorze stron.

💡 *Materiały od tej lekcji dostajesz w `.md` w repozytorium zajęć. Pliki `.docx`, które masz do tej pory, to ostatnie – dokładnie z powodów wyżej.*

1.1 Gdzie go użyjesz

- `README.md` – każde repozytorium
- opisy Issues i Pull Requestów na GitHubie
- `CLAUDE.md` / `AGENTS.md` / `GEMINI.md` – instrukcje dla agentów kodujących
- notatki, dokumentacja projektu, wiki, blogi techniczne
- odpowiedzi LLM – to, co widzisz w czacie, to renderowany Markdown

2. Składnia – to, co naprawdę potrzebne

2.1 Nagłówki

```
# Tytuł dokumentu (h1) – jeden na plik
## Rozdział (h2)
### Podrozdział (h3)
```

Spacja po # jest obowiązkowa. Nie skacz poziomami (po # nie dawaj od razu ###).

2.2 Akapity i podział linii

Akapit = tekst oddzielony **pustą linią**. Pojedyncze wciśnięcie Enter nie robi nowej linii – Markdown skleja. Chcesz łamanie w akapicie: dwie spacje na końcu linii albo po prostu nowy akapit.

2.3 Wyróżnienia

```
**pogrubienie**
*kursywa*
~~przekreślenie~~
`kod w linii` – nazwy plików, komend, zmiennych
```

💡 *Wszystko, co jest nazwą z kodu – app.py, git push, Product, /products/5 – w odwrotnych apostrofach Odróżnia się od zwykłego tekstu i nie da się pomylić z prozą.*

2.4 Listy

```
- element
- element
  - zagnieżdżony (2 spacje wcięcia)

1. pierwszy
2. drugi
3. trzeci

- [ ] zadanie do zrobienia
- [x] zadanie zrobione
```

Listę numerowaną możesz pisać samymi 1. – Markdown sam ponumeruje. Checkboxy [] / [x] działają na GitHubie jako klikalna lista (Issues, PR, README).

2.5 Kod

Blok kodu: trzy odwrotne apostrofy, nazwa języka, kod, trzy apostrofy. Nazwa języka włącza kolorowanie składni.

```
```python
@app.route("/products")
def product_list():
 return render_template("products/list.html")
```

```bash
pip install -r requirements.txt
python app.py
```

```html
<form method="post">...</form>
```
```

Języki, które będziesz podawać: `python`, `bash`, `powershell`, `html`, `css`, `javascript`, `json`, `sql`, `kotlin`, `text`.

⚠ Blok kodu bez nazwy języka działa, ale bez kolorów. Z błędną nazwą – też działa. Zawsze podawaj język.

2.6 Linki i obrazy

```
[tekst linku](https://flask.palletsprojects.com)
[plik w repo](docs/instalacja.md)
[sekcja w tym pliku](#uruchomienie)

![opis obrazka](screenshots/lista.png)
```

Linki względne (`docs/instalacja.md`, `screenshots/lista.png`) działają na GitHubie i w edytorze – plik musi być w repo. Zrzuty ekranu trzymaj w folderze `screenshots/` lub `docs/img/`.

2.7 Tabele

| Metoda | Adres | Opis |
|--------|---------------|-----------|
| GET | /products | lista |
| POST | /products/add | dodawanie |

Linia z `---` jest obowiązkowa – oddziela nagłówki. Wyrównywanie kolumn spacjami jest opcjonalne (ładniej w źródle, bez wpływu na wynik). Dwukropek w `|-:|`, `|-:|`, `|-:|` wyrównuje do lewej / prawej / środka.

2.8 Cytat i linia pozioma

```
> Uwaga: baza jest w instance/, nie commituj jej.
---
```

Cytat `>` przydaje się na ostrzeżenia i uwagi. `---` w osobnej linii to pozioma kreska.

3. README – szablon na projekt

To, co powinno być w każdym Twoim repozytorium od teraz:

```
# Nazwa projektu

Jedno-dwa zdania: co to jest i dla kogo.

## Technologie

- Python 3.12, Flask 3, Flask-SQLAlchemy
- SQLite
- Jinja2, CSS

## Uruchomienie

```powershell
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Aplikacja: http://localhost:5000

## Struktura

```text
app.py - konfiguracja i widoki
models.py - modele SQLAlchemy
templates/ - szablony Jinja2
static/ - CSS
```

## Trasy



Metoda	Adres	Opis
GET	/products	lista
GET	/products/<id>	szczegóły
GET/POST	/products/add	dodawanie
POST	/products/<id>/delete	usuwanie



## Status

- [x] model Product
- [x] lista, szczegóły, dodawanie
- [ ] edycja
- [ ] wyszukiwarka

## Autor

Imię Nazwisko, 4TP, 2026/2027
```

💡 *Podgląd na żywo: w VS Code Ctrl + Shift + V, w PyCharm zakładka Preview obok edytora. Na GitHubie – po prostu otwórz plik.*

4. Zasady

1. Jeden # na plik – to tytuł. Reszta ## i ###.
2. Pusta linia przed i po nagłówku, liście, bloku kodu, tabeli. Bez tego niektóre renderery skleją.
3. Nazwy z kodu w odwrotnych apostrofach (klawisz pod Esc). Zawsze.
4. Blok kodu zawsze z nazwą języka.
5. Nie wklejaj zrzutu ekranu z kodem – wklej kod. Zrzut ekranu tylko na UI.
6. Nie używaj HTML-a w Markdown, chyba że naprawdę nie da się inaczej.
7. Plik .md to tekst – obowiązuje go to samo, co kod: po angielsku nazwy, commit docs:

5. Zadania

Zadanie 1: README od nowa

Przepisz README swojego projektu Flask według szablonu z rozdziału 3. Wszystkie sekcje. Tabela tras z prawdziwymi adresami. Podgląd w edytorze, potem sprawdź na GitHubie. Commit: `docs: rewrite README`.

Zadanie 2: Notatki z materiału 08

Plik `docs/notes-http.md` w repo: tabela kodów HTTP, które widziałeś (kod, nazwa, gdzie), i lista metod HTTP z jednym zdaniem każda. Blok kodu z przykładem `abort(404)`. Link do tego pliku z README.

Zadanie 3: Zrzut ekranu

Folder `screenshots/`, jeden zrzut listy produktów. Wstaw do README pod sekcją „Status” przez `![...](...)`. Sprawdź, że wyświetla się na GitHubie.

Zadanie 4: Issue

Na GitHubie załóż Issue „Dodać edycję produktu” z opisem w Markdown: co ma robić (lista), jakie trasy (tabela), lista kontrolna `[ ]` z krokami. To jest Twoje pierwsze zadanie w trackerze – zamkniesz je commitem, gdy edycja będzie gotowa.

Zadanie 5: Plik dla agenta

`AGENTS.md` w głównym folderze: nagłówek z nazwą projektu, lista technologii, 3–5 zasad (angielskie nazwy, ruff przed commitem, docstringi, bez nowych pakietów bez pytania). Uruchom agenta i zapytaj „jakie zasady obowiązują w tym projekcie?” – ma je powtórzyć.